

Caso Prácticas IPC 2026

Running la Safor

Interfaces Persona Computador · IPC

Contenido

1.	Introducción	4
2.	Caso de estudio	4
	Categorías de escenarios	4
3.	Escenarios sobre el usuario	5
3.1	Registrarse en la aplicación	5
3.2	Autenticarse	5
3.3	Modificar perfil.....	5
3.4	Cerrar sesión	6
3.5	Visualizar historial de sesiones	6
4.	Escenarios sobre las actividades	6
4.1	Registrar actividad nueva.....	6
4.2	Añadir anotaciones a la actividad	6
4.3	Visualizar una actividad	6
4.4	Acumulado de actividades	7
5.	Escenarios sobre la ruta en el mapa.....	7
5.1	Realizar zoom	7
6.	Escenarios adicionales	7
6.1	Perfil de desnivel.....	7
6.2	Velocidad sobre el trazado.....	7
7.	Escenarios sobre mapas.....	7
7.1	Añadir mapa al sistema	7
8.	Recursos para realizar la práctica.....	8
8.1	Los mapas	8
8.2	Los ficheros GPX.....	9
8.3	Modelo de datos y clases de la librería.....	10
8.3.1	User	11
8.3.2	Session	12
8.3.3	Activity	12
8.3.4	TrackPoint.....	13
8.3.5	Annotation.....	13
8.3.6	AnnotationType.....	14
8.3.7	GeoPoint.....	14
8.3.8	MapRegion	15
8.3.9	MapProjection	15
8.3.10	GeoUtils	16
8.4	La clase SportActivityApp	17

8.4.1	Gestión de usuarios	17
8.4.2	Gestión de actividades.....	17
8.4.3	Gestión de anotaciones	17
8.4.4	Gestión de mapas.....	18
8.4.5	Sesiones	18
9.	Ejemplo de uso de la librería y estructura del proyecto.....	19
9.1	Ejemplo de uso	19
9.2	Estructura del proyecto a desarrollar	19
9.3	Añadir las librerías en NetBeans.....	20
10.	Persistencia de los datos.....	20
11.	Proyecto base.....	21
12.	Instrucciones de entrega	21

1. Introducción

Este documento describe el caso de estudio que deberéis implementar como proyecto de la asignatura. Para centrar el trabajo en el diseño de la interfaz y la interacción con el usuario, se proporciona una librería (IPC2026.jar) que se encarga de toda la parte técnica: modelo de datos, persistencia en base de datos SQLite, parseo de ficheros GPX y cálculos geográficos.

El documento se organiza de la siguiente manera:

- Caso de estudio: descripción del sistema a desarrollar.
- Escenarios de uso: situaciones concretas que debe soportar la aplicación.
- Recursos: descripción detallada de las clases de la librería.
- Proyecto base, persistencia e instrucciones de entrega.

Importante: La librería gestiona automáticamente la base de datos, la importación de GPX y los cálculos. El alumno NO debe modificar ni reimplementar estas clases, solo utilizarlas.

2. Caso de estudio

El club Running la Safor quiere desarrollar una aplicación para que sus socios puedan registrar sus actividades al aire libre y hacer un seguimiento y análisis de las mismas. Los socios, tras registrarse, podrán subir sus actividades a través de ficheros GPX generados por relojes, dispositivos wearables o aplicaciones de tracking en el móvil.

Tras subir el fichero, la aplicación permitirá visualizar el trazado sobre un mapa, añadir anotaciones geográficas, ver el perfil de desnivel y consultar estadísticas como distancia, duración, velocidad y desnivel.

La aplicación por desarrollar también facilitará ciertas tareas de configuración o de control de la aplicación que se detallan posteriormente.

Seguidamente, se detallan los escenarios de uso que se han obtenido tras el análisis de requisitos del sistema y que deben utilizarse para realizar el diseño conceptual y después diseñar e implementar adecuadamente la aplicación requerida. Estos escenarios se han agrupado en varias categorías.

Categorías de escenarios

- Categoría 1 — Usuarios: registrarse, autenticarse, modificar perfil, cerrar sesión, historial de sesiones.
- Categoría 2 — Actividades: importar GPX, añadir anotaciones, visualizar actividad, acumulado, borrar actividad.

- Categoría 3 — Ruta en mapa: zoom in/out, añadir anotaciones.
- Categoría 4 — Otras: perfil de desnivel, velocidad sobre el trazado.
- Categoría 5 — Mapas: añadir un mapa al sistema.

3. Escenarios sobre el usuario

3.1 Registrarse en la aplicación

Juan quiere registrarse en la aplicación. Tras instalarla accede a la opción de registro. El sistema solicita: nickname (identificador único), correo electrónico, contraseña, fecha de nacimiento y, opcionalmente, un avatar.

Juan introduce el nickname "jgarcia", pero el sistema le informa de que ya está en uso. Elige "jpgarcia". Introduce la contraseña "passPER21!" y el correo "jgarcia@gmail.com", con fecha de nacimiento 14/03/2005. Decide no subir avatar en ese momento.

El sistema valida los datos según las siguientes reglas y registra al usuario informándole del resultado:

- Nickname: entre 6 y 15 caracteres, solo letras, dígitos, guion o subguion.
- Contraseña: entre 8 y 20 caracteres, con al menos una mayúscula, una minúscula, un dígito y un símbolo (!@#\$%&*()-+=).
- Email: formato válido usuario@dominio.
- Edad mínima: el usuario debe tener más de 12 años.

Nota de implementación: Usa los métodos estáticos `User.checkNickName()`, `User.checkEmail()`, `User.checkPassword()` e `User.isOlderThan()` para validar cada campo antes de llamar a `app.registerUser()`.

3.2 Autenticarse

Juan abre la aplicación y selecciona la opción de autenticarse. El sistema solicita nickname y contraseña. Juan introduce "jpgarcia" y su contraseña. El sistema verifica las credenciales y, si son correctas, da acceso al resto de funciones.

3.3 Modificar perfil

Juan decide cambiar su avatar por una imagen de su equipo. Accede a la opción de modificar perfil. El sistema muestra la información actual: nickname (no editable), correo, contraseña y fecha de nacimiento. Juan selecciona una imagen de su máquina como nuevo avatar y solicita guardar los cambios.

El sistema aplica las mismas reglas de validación que en el registro y, si todo es correcto, actualiza los datos.

Nota de implementación: Usa `app.updateCurrentUser(email, password, birthDate, avatarPath)`. Si el campo de contraseña se deja en blanco, mantén la contraseña actual (`user.getPassword()`) sin modificarla.

3.4 Cerrar sesión

Juan decide cerrar la aplicación. El sistema guarda automáticamente los datos de la sesión: instante de inicio y fin, número de actividades importadas, número de actividades visualizadas y número de anotaciones creadas. Estos datos quedan disponibles en el historial.

3.5 Visualizar historial de sesiones

Juan accede al historial de sesiones para analizar su uso de la aplicación. El sistema muestra las sesiones registradas con su duración y estadísticas de uso (actividades importadas, vistas y anotaciones creadas), así como totales acumulados.

4. Escenarios sobre las actividades

4.1 Registrar actividad nueva

Juan selecciona un fichero GPX de una carrera del fin de semana. El sistema lo procesa, muestra la ruta sobre el mapa resaltando el inicio (verde) y el final (rojo), y presenta las estadísticas: distancia, duración, velocidad media, ritmo medio, desnivel positivo y negativo, altitud mínima y máxima.

4.2 Añadir anotaciones a la actividad

Juan recuerda dos zonas peligrosas del recorrido y quiere anotarlas. Selecciona la actividad, hace clic derecho sobre el mapa en el punto correspondiente y elige el tipo de anotación (punto, texto, círculo o línea). Escribe el texto y elige el color. Las anotaciones quedan persistentes y se muestran al visualizar la actividad.

4.3 Visualizar una actividad

Juan selecciona una actividad de la lista. El sistema muestra el trazado sobre el mapa, las anotaciones existentes y las estadísticas calculadas.

4.4 Acumulado de actividades

Juan consulta el total de actividades del mes: tiempo total, distancia acumulada, metros de ascenso y descenso totales.

5. Escenarios sobre la ruta en el mapa

5.1 Realizar zoom

Juan amplía o reduce la vista del mapa mediante los botones de zoom (+/-). El trazado y las anotaciones se escalan con el mapa.

6. Escenarios adicionales

6.1 Perfil de desnivel

Junto al mapa se muestra una gráfica con el perfil de altitud de la actividad (eje X: distancia en km, eje Y: altitud en metros). Al pasar el ratón sobre la gráfica se destaca el punto correspondiente sobre el mapa.

6.2 Velocidad sobre el trazado

El sistema permite visualizar la velocidad seguida en cada tramo del recorrido, codificada visualmente sobre el trazado del mapa.

7. Escenarios sobre mapas

7.1 Añadir mapa al sistema

Juan ha participado en una carrera en Montanejos, pero no tiene ese mapa en la aplicación. Accede a la gestión de mapas e introduce el fichero JPG del mapa junto con las coordenadas geográficas de su bounding box (latitud y longitud mínima y máxima). Estas coordenadas las obtiene del script Python proporcionado, que genera la imagen y las imprime por pantalla.

Importante: Los valores del bounding box deben coincidir exactamente con los que imprime el script. Si se usan valores aproximados, la proyección de coordenadas GPS a píxeles será incorrecta. El script se deja como material adicional al proyecto y si se ejecuta es al margen de la aplicación a desarrollar.

8. Recursos para realizar la práctica

Para centrar la práctica en el diseño de la interfaz y la interacción, se proporciona la librería IPC2026.jar, que implementa el modelo de datos y proporciona toda la funcionalidad de soporte. El alumno debe utilizar las clases ya implementadas, sin reimplementarlas.

La librería gestiona automáticamente:

- La creación de la base de datos SQLite si no existe (fichero sportactivity.db en el directorio del proyecto).
- El almacenamiento y recuperación de usuarios, sesiones, actividades, anotaciones y mapas.
- La importación y parseo de ficheros GPX.
- Los cálculos de distancia, duración, desnivel y velocidad.
- La transformación de coordenadas GPS a píxeles de mapa.

8.1 Los mapas

Para mostrar mapas y rutas sobre estos vamos a utilizar una aproximación bastante sencilla, pero a la vez lo suficientemente realista para obtener una aplicación de aspecto profesional. Hemos evitado también otras alternativas basadas en librerías externas para facilitar el desarrollo del proyecto.

La estrategia para mostrar mapas consiste en usar una imagen estática como fondo y dibujar el trazado encima. Cada imagen tiene asociadas unas coordenadas geográficas (bounding box) que permiten transformar posiciones GPS en píxeles.

En lugar de tener una sola imagen o mapa vamos a disponer de varias de estas imágenes y escogeremos la adecuada a la ruta que queramos visualizar. Cada uno de estos mapas o imágenes tendrán asociado una latitud y longitud mínima y máxima, lo que va a permitir transformar las posiciones GPS en píxeles del mapa.

La librería incluye tres mapas por defecto con sus imágenes y coordenadas ya registradas:

Región	Lat mín	Lat máx	Lon mín	Lon máx
Valencia	39.334297	39.554883	-0.505371	-0.285645
Sierra Calderona	39.588757	39.757880	-0.571289	-0.285645
Pirineos	42.585444	42.811522	0.000000	0.307617

Atención: Estos valores exactos son los que usa la librería internamente. Si añades mapas propios, usa siempre las coordenadas que imprime el script `generar_mapas_hd.py`, nunca valores aproximados.

El código básico para mostrar el mapa y la ruta en JavaFX es el siguiente:

```
// Cargar la imagen del mapa
Image map = new Image(new File(region.getImagePath()).toURI().toString());
ImageView mapView = new ImageView(map);

// Crear el Pane contenedor del mapa
Pane mapPane = new Pane(mapView);
mapPane.setPrefSize(map.getWidth(), map.getHeight());

// Crear la proyección para convertir GPS → píxeles
MapProjection proj = new MapProjection(region, map.getWidth(), map.getHeight());

// Dibujar la ruta
Polyline route = new Polyline();
for (TrackPoint tp : activity.getTrackPoints()) {
    Point2D p = proj.project(tp);
    route.getPoints().addAll(p.getX(), p.getY());
}
mapPane.getChildren().add(route);
```

8.2 Los ficheros GPX

Un fichero **GPX** (GPS Exchange Format) es un estándar abierto basado en lenguaje XML diseñado para el intercambio de datos GPS entre aplicaciones y dispositivos.

Un archivo GPX suele contener tres tipos de datos fundamentales: Waypoint, Routes y Tracks, estos últimos son lo que nos interesan en este proyecto

- Tracks (tracks o rastreos): El registro real de un recorrido ya realizado, que incluye tiempo y elevación en cada punto

También suele venir acompañado de metadatos como el nombre, que también resultará de interés para el proyecto.

Hay otros datos como el ritmo cardiaco, temperatura, etc., que no vamos a tener en cuenta en este proyecto.

Para extraer estos datos el fichero GPX vamos a utilizar un parser de XML estándar, para guardar cada uno de los puntos que obtengamos del track vamos a crear objetos de la clase **TrackPoint** (esta clase se describe en apartados posteriores).

8.3 Modelo de datos y clases de la librería

Tras el estudio de los objetos de tarea (diseño conceptual) realizado sobre los escenarios se ha decidido implementar las siguientes clases en el modelo:

- User
- Session
- Activity
- Annotation

Como hemos indicado en apartados anteriores, además de las clases obtenidas a partir de los escenarios en el proceso de diseño conceptual, tenemos que añadir las clases que nos permiten trabajar con el modelo de mapas que hemos presentado y con los datos GPS. Estas clases son:

- TrackPoint
- GeoPoint
- MapRegion
- MapProjection
- GeoUtils
- SportActivityApp

Las clases del modelo son creadas exclusivamente por la librería. El alumno no puede instanciarlas directamente (sus constructores son package-private). Solo se pueden instanciar directamente:

- GeoPoint — para definir el punto geográfico de una anotación.
- Annotation — para crear nuevas anotaciones antes de añadirlas a la actividad.
- MapProjection — para convertir coordenadas en una vista de mapa.

Regla general: Todas las demás clases (Activity, TrackPoint, User, Session, MapRegion) se obtienen exclusivamente a través de los métodos de SportActivityApp, nunca con new.

8.3.1 User

Almacena la información de un usuario. El nickname no puede modificarse tras el registro.

Método	Descripción
<code>getNickName()</code>	Retorna el nombre de usuario (no modificable).
<code>getEmail()</code> / <code>setEmail(email)</code>	Recupera o modifica el correo electrónico.
<code>getPassword()</code> / <code>setPassword(pass)</code>	Recupera o modifica la contraseña.
<code>getAvatarPath()</code> / <code>setAvatarPath(path)</code>	Recupera o modifica la ruta del fichero de avatar.
<code>getAvatar()</code>	Carga y retorna el avatar como <code>javafx.scene.image.Image</code> . Retorna null si no hay avatar.
<code>getBirthDate()</code> / <code>setBirthDate(date)</code>	Recupera o modifica la fecha de nacimiento.
<code>getActivities()</code>	Retorna la lista de actividades del usuario.
<code>getSessions()</code>	Retorna la lista de sesiones del usuario.

Importante — avatar: El avatar se almacena como ruta de fichero (String `avatarPath`), NO como objeto `Image`. Usa `getAvatarPath()` para guardar/recuperar la ruta, y `getAvatar()` para obtener la `Image` cuando quieras mostrarla.

Métodos estáticos de validación

Método	Descripción
<code>User.checkNickName(nick)</code>	True si el nickname cumple las reglas (6-15 chars, letras/dígitos/guion/subguion).
<code>User.checkEmail(email)</code>	True si el email tiene formato válido.
<code>User.checkPassword(pass)</code>	True si la contraseña cumple las reglas (8-20 chars, mayúscula, minúscula, dígito, símbolo).
<code>User.isOlderThan(birthDate, years)</code>	True si la persona tiene estrictamente más de 'years' años.

8.3.2 Session

Representa una sesión de uso. Se crea automáticamente al cerrar sesión. Sus atributos no pueden modificarse.

Método	Descripción
<code>getStartTime()</code> / <code>getEndTime()</code>	Instante de inicio y fin de la sesión.
<code>getDuration()</code>	Duración total de la sesión como <code>Duration</code> .
<code>getImportedActivities()</code>	Número de actividades importadas durante la sesión.
<code>getViewedActivities()</code>	Número de actividades visualizadas durante la sesión.
<code>getAnnotationsCreated()</code>	Número de anotaciones creadas durante la sesión.

8.3.3 Activity

Representa una actividad deportiva importada desde un fichero GPX.

Método	Descripción
<code>getId()</code>	Identificador único en la base de datos.
<code>getName()</code> / <code>setName(name)</code>	Recupera o modifica el nombre. Para persistir el cambio usa <code>app.renameActivity()</code> .
<code>getUser()</code>	Usuario propietario de la actividad.
<code>getStartTime()</code> / <code>getEndTime()</code>	Fecha y hora de inicio y fin (del primer y último <code>TrackPoint</code>).
<code>getTrackPoints()</code>	Lista no modificable de puntos GPS en orden de registro.
<code>getAnnotations()</code>	Lista no modificable de anotaciones asociadas.
<code>getSuggestedMap()</code>	<code>MapRegion</code> recomendada para representar la actividad.
<code>getStartPoint()</code> / <code>getEndPoint()</code>	Primer y último <code>TrackPoint</code> del recorrido.

<code>getTotalDistance()</code>	Distancia total en metros (fórmula Haversine).
<code>getDuration()</code>	Duración total como Duration.
<code>getAverageSpeed()</code>	Velocidad media en km/h.
<code>getAveragePace()</code>	Ritmo medio en minutos por kilómetro.
<code>getElevationGain()</code> <code>getElevationLoss()</code>	/ Desnivel positivo y negativo acumulados en metros.
<code>getMinElevation()</code> <code>getMaxElevation()</code>	/ Altitud mínima y máxima registradas en metros.

8.3.4 TrackPoint

Representa un punto del recorrido GPS almacenado en el fichero GPX.

Método	Descripción
<code>getLatitude()</code> / <code>getLongitude()</code>	Coordenadas geográficas en grados decimales (WGS-84).
<code>getElevation()</code>	Altitud sobre el nivel del mar en metros.
<code>getTime()</code>	Instante UTC asociado al punto. Puede ser null.
<code>distanceTo(other)</code>	Distancia en metros hasta otro TrackPoint (Haversine).
<code>timeTo(other)</code>	Diferencia temporal entre dos puntos como Duration.
<code>speedTo(other)</code>	Velocidad media entre dos puntos en km/h.

8.3.5 Annotation

Representa una anotación del usuario sobre el mapa. Es la única clase del modelo que el alumno puede instanciar directamente (sin id):

```
// Constructor público (sin id — la librería lo asigna al guardar):
Annotation ann = new Annotation(
    AnnotationType.POINT,    // tipo de anotación
    "Zona peligrosa",        // texto (puede ser vacío)
    "#E74C3C",               // color en formato CSS hex
    2.0,                      // grosor del trazo
    List.of(geoPoint)        // puntos geográficos (ver tabla de tipos)
);
// Persistir y obtener la anotación con id asignado:
```

```
Annotation saved = app.addAnnotation(activity, ann);
```

Método	Descripción
<code>getId()</code>	Identificador en BD (0 si aún no se ha persistido).
<code>getType()</code>	Tipo de anotación (AnnotationType).
<code>getText()</code> / <code>setText(text)</code>	Texto asociado a la anotación.
<code>getColor()</code> / <code>setColor(color)</code>	Color en formato CSS hex, p.ej. "#E74C3C".
<code>getStrokeWidth()</code> / <code>setStrokeWidth(w)</code>	Grosor del trazo en píxeles.
<code>getGeoPoints()</code>	Lista no modificable de puntos geográficos.

8.3.6 AnnotationType

Enumeración con los tipos de anotación disponibles. Cada tipo requiere un número específico de GeoPoints:

Valor	Descripción	GeoPoints necesarios
<code>POINT</code>	Círculo relleno con etiqueta de texto opcional.	1 (posición)
<code>TEXT</code>	Etiqueta de texto anclada a una posición.	1 (ancla)
<code>LINE</code>	Segmento recto entre dos puntos.	2 (inicio, fin)
<code>CIRCLE</code>	Circunferencia definida por centro y borde.	2 (centro, borde)

8.3.7 GeoPoint

Coordenada geográfica inmutable. Es uno de los tres tipos que el alumno puede instanciar directamente.

```
// Constructor público:
GeoPoint punto = new GeoPoint(latitud, longitud);

// Obtener a partir de un click en el mapa:
GeoPoint geo = projection.unproject(e.getX(), e.getY());
```

Método	Descripción
<code>getLatitude()</code>	Latitud en grados decimales.
<code>getLongitude()</code>	Longitud en grados decimales.

8.3.8 MapRegion

Representa una región geográfica asociada a una imagen de mapa. Solo puede obtenerse a través de SportActivityApp (constructor package-private).

Método	Descripción
<code>getName()</code>	Nombre de la región, p.ej. "Valencia".
<code>getImagePath()</code>	Ruta relativa a la imagen, p.ej. "maps/valencia.jpg".
<code>getLatMin()</code> / <code>getLatMax()</code>	Límites sur y norte del bounding box en grados decimales.
<code>getLonMin()</code> / <code>getLonMax()</code>	Límites oeste y este del bounding box en grados decimales.
<code>contains(lat, lon)</code>	True si la coordenada está dentro del bounding box.
<code>contains(activity)</code>	True si todos los puntos de la actividad están en el bounding box.

8.3.9 MapProjection

Convierte entre coordenadas geográficas y píxeles de imagen usando proyección Web Mercator. El alumno crea una instancia por cada vista de mapa activa.

```
// Constructor (alumno puede instanciar directamente):
MapProjection proj = new MapProjection(region, imgWidth, imgHeight);
```

Método	Descripción
<code>project(lat, lon)</code>	Convierte coordenada geográfica → Point2D (píxeles en la imagen).
<code>project(GeoPoint p)</code>	Versión para GeoPoint.
<code>project(TrackPoint p)</code>	Versión para TrackPoint.
<code>projectActivity(activity)</code>	Convierte todos los TrackPoints de una actividad → List<Point2D>.
<code>unproject(screenX, screenY)</code>	Convierte píxeles del Pane → GeoPoint. Pasa e.getX()/e.getY() directamente; JavaFX ya corrige el zoom.

```
getRegion()
```

Retorna la MapRegion asociada a esta proyección.

Uso de unproject(): Llama a `projection.unproject(e.getX(), e.getY())` si quieres transformar las coordenadas del ratón. JavaFX corrige automáticamente las coordenadas según el nivel de zoom, por lo que no necesitas escalar manualmente.

8.3.10 GeoUtils

Clase de utilidad con métodos estáticos para cálculos geográficos. Todos los cálculos de distancia usan la fórmula Haversine (error < 0.5% para distancias típicas de deporte).

Método	Descripción
<code>GeoUtils.distance(lat1,lon1,lat2,lon2)</code>	Distancia entre dos coordenadas en metros.
<code>GeoUtils.distance(GeoPoint, GeoPoint)</code>	Distancia entre dos GeoPoints en metros.
<code>GeoUtils.distance(TrackPoint, TrackPoint)</code>	Distancia entre dos TrackPoints en metros.
<code>GeoUtils.totalDistance(List<TrackPoint>)</code>	Distancia total del recorrido en metros.
<code>GeoUtils.elevationGain(List<TrackPoint>)</code>	Desnivel positivo acumulado en metros.
<code>GeoUtils.elevationLoss(List<TrackPoint>)</code>	Desnivel negativo acumulado en metros (valor positivo).
<code>GeoUtils.bearing(GeoPoint, GeoPoint)</code>	Rumbo desde p1 a p2 en grados (0-360, sentido horario desde el norte).

8.4 La clase SportActivityApp

Esta es la clase principal de la librería. Implementa el patrón Singleton: solo existe una instancia durante la ejecución. Agrupa todos los métodos necesarios para gestionar usuarios, actividades, anotaciones, mapas y sesiones.

```
// Obtener la instancia (siempre así, nunca con new):
SportActivityApp app = SportActivityApp.getInstance();
```

8.4.1 Gestión de usuarios

Método	Descripción
<code>registerUser(nick, email, pass, birthDate, avatarPath)</code>	Registra un nuevo usuario. Valida todos los campos. Retorna true si tiene éxito.
<code>login(nickName, password)</code>	Autentica un usuario e inicia la sesión. Retorna true si las credenciales son correctas.
<code>logout()</code>	Cierra la sesión activa y guarda automáticamente las estadísticas de sesión en la BD.
<code>getCurrentUser()</code>	Retorna el User autenticado actualmente, o null si no hay sesión.
<code>updateCurrentUser(email, pass, birthDate, avatarPath)</code>	Actualiza el perfil del usuario autenticado. Valida todos los campos.

8.4.2 Gestión de actividades

Método	Descripción
<code>importActivity(File gpxFile)</code>	Parsea un fichero GPX, almacena la actividad en la BD y retorna el objeto Activity creado.
<code>getUserActivities()</code>	Retorna las actividades del usuario autenticado, ordenadas por fecha descendente.
<code>getActivitiesByUser(User)</code>	Retorna las actividades de cualquier usuario.
<code>removeActivity(Activity)</code>	Elimina una actividad y todos sus datos de la BD.
<code>renameActivity(Activity, String)</code>	Cambia el nombre de una actividad y lo persiste.

8.4.3 Gestión de anotaciones

Método	Descripción
<code>addAnnotation(Activity, Annotation)</code>	Guarda una anotación en la BD. Retorna la Annotation con el id asignado, o null si falla.
<code>removeAnnotation(Annotation)</code>	Elimina una anotación de la BD.
<code>updateAnnotationColor(Annotation)</code>	Persiste el cambio de color de una anotación existente.

8.4.4 Gestión de mapas

Método	Descripción
<code>getMapRegions()</code>	Retorna todas las regiones de mapa registradas, ordenadas por nombre.
<code>getUnusedMapRegions()</code>	Retorna solo las regiones que no están referenciadas por ninguna actividad (candidatas a borrar).
<code>findMapForActivity(Activity)</code>	Encuentra la región que mejor cubre el trazado de la actividad.
<code>findMapForPoint(lat, lon)</code>	Encuentra la región que contiene una coordenada geográfica.
<code>addMapRegion(name, imageFile, latMin, latMax, lonMin, lonMax)</code>	Registra una nueva región: copia la imagen a maps/ e inserta el registro. Retorna la MapRegion creada o null si falla.
<code>removeMapRegion(MapRegion)</code>	Borra una región si ninguna actividad la referencia. También borra el fichero de imagen (salvo los tres mapas por defecto).

8.4.5 Sesiones

Método	Descripción
<code>getSessionsByUser(User)</code>	Retorna las sesiones del usuario, ordenadas por fecha descendente.

9. Ejemplo de uso de la librería y estructura del proyecto

9.1 Ejemplo de uso

```
SportActivityApp app = SportActivityApp.getInstance();

// 1. Autenticación
boolean ok = app.login("jpgarcia", "passPER21!");

if (ok) {
    // 2. Importar actividad desde fichero GPX
    Activity activity = app.importActivity(new File("ruta.gpx"));

    // 3. Obtener mapa adecuado y crear proyección
    MapRegion region = activity.getSuggestedMap();
    Image img = new Image(new File(region.getImagePath()).toURI().toString());
    MapProjection proj = new MapProjection(region, img.getWidth(),
    img.getHeight());

    // 4. Dibujar el trazado
    Polyline route = new Polyline();
    for (TrackPoint tp : activity.getTrackPoints()) {
        Point2D p = proj.project(tp);
        route.getPoints().addAll(p.getX(), p.getY());
    }
    mapPane.getChildren().add(route);

    // 6. Mostrar estadísticas
    System.out.printf("Distancia: %.2f km%n",
    activity.getTotalDistance()/1000.0);
    System.out.printf("Duración: %s%n", activity.getDuration());
    System.out.printf("Desnivel+: %.0f m%n", activity.getElevationGain());

    // 7. Cerrar sesión (guarda estadísticas automáticamente)
    app.logout();
}
```

9.2 Estructura del proyecto a desarrollar

El directorio de trabajo de NetBeans es la carpeta raíz del proyecto. La librería crea y busca todos sus ficheros relativos a esa carpeta. La siguiente estructura muestra cómo deben organizarse los recursos necesarios para que la aplicación funcione correctamente.

MiProyecto/	<- raíz del proyecto NetBeans
├── lib/	<- librerías externas
│ ├── IPC2026.jar	<- librería proporcionada
│ └── sqlite-jdbc-x.x.x.jar	<- SQLite JDBC (descargar de Maven)
├── maps/	<- imágenes de mapa
│ ├── valencia.jpg	<- copiar antes de ejecutar
│ ├── calderona.jpg	<- copiar antes de ejecutar
│ └── pirineos.jpg	<- copiar antes de ejecutar
└── avatars/	<- creada automáticamente al registrar usuarios

```
└─ sportactivity.db          <- creada automáticamente al arrancar
```

Importante: Las carpetas avatars/ y el fichero sportactivity.db se crean automáticamente la primera vez que se ejecuta la aplicación. La carpeta maps/ y los tres ficheros JPG deben copiarse manualmente antes de la primera ejecución. Si los mapas no están presentes, la librería registra las regiones en la base de datos pero no puede mostrar las imágenes.

9.3 Añadir las librerías en NetBeans

Para añadir IPC2026.jar y sqlite-jdbc al proyecto:

1. Clic derecho sobre el proyecto -> Properties -> Libraries
2. En la pestaña Compile, pulsar 'Add JAR/Folder'
3. Seleccionar lib/IPC2026.jar
4. Repetir para lib/sqlite-jdbc-x.x.x.jar

La librería SQLite JDBC puede descargarse desde poliformat o desde la siguiente dirección <https://mvnrepository.com/artifact/org.xerial/sqlite-jdbc>. Descargad el fichero .jar y guardadlo en la carpeta lib/ del proyecto.

Versión SQLite: Usad la versión 3.45 o superior. La librería IPC2026.jar ha sido compilada y probada con sqlite-jdbc-3.45.x.jar.

10. Persistencia de los datos

La librería utiliza una base de datos SQLite que se crea automáticamente en el directorio de trabajo del proyecto con el nombre sportactivity.db. No es necesario configurar nada; la librería inicializa las tablas al arrancar.

Si necesitáis inspeccionar el contenido de la base de datos durante el desarrollo, podéis usar la aplicación gratuita DB Browser for SQLite (<https://sqlitebrowser.org>).

Se proporcionará una base de datos con un usuario genérico para poder probar la aplicación antes de tener implementado el registro de usuarios.

11. Proyecto base

Se proporciona el proyecto NetBeans MapaDemo como punto de partida. Este proyecto muestra cómo implementar el zoom sobre el mapa usando un Group dentro de un ScrollPane, que es la estructura necesaria para que el escalado funcione correctamente con los nodos superpuestos.

Recomendación: Usad este proyecto como base, eliminando o adaptando lo que no sea necesario para vuestra implementación. La estrategia adecuada pasa por generar cada vista en un fichero fxml diferente e ir cambiando las diferentes áreas del BorderPane según las acciones del usuario.

12. Instrucciones de entrega

La entrega se realizará en grupos de 4 alumnos del mismo grupo de laboratorio. El proceso de entrega es el siguiente:

- Realizad el diseño conceptual y los prototipos de baja fidelidad antes de implementar.
- Revisad los prototipos con vuestro profesor de prácticas o teoría.
- Cread una cuenta en GitHub y subid el código y la documentación de forma incremental.
- GitHub registra qué cambios ha realizado cada integrante — esto forma parte de la evaluación.
- La entrega final se realiza a través de una tarea de poliformat.

Importante: Solo se aceptará un grupo con menos de 4 integrantes por grupo de laboratorio, en caso de que no haya alumnos suficientes para completar todos los grupos.